



TECHNISCHE UNIVERSITÄT
CHEMNITZ

AUTOSAR Application Development

Seminar Report

Chair of Computer Engineering

Dept. of Computer Science

Submitted by: Mohammad Younes

Matrikel Nr.: 432674

Submission date: 03.03.2017

Supervising tutor: Mr. René Schmidt

Abstract

The usage of E/E systems in vehicles increases rapidly. These systems are very complex, a modern car may have around 100 ECUs that communicate with each other via buses, and around one million line of code. E/E systems are safety critical systems in which software plays a crucial role. Software ensures reliable and fast reaction of these embedded systems in the vehicle. The software for ECUs is very complex and hardware dependent which makes the development process of such a software time consuming and very costly.

AUTOSAR makes the ECU application software hardware independent and enables reusability of similar software components which shall reduce effort, time, and cost of the software development process.

This report discusses AUTOSAR architecture and its layers, the software development process for vehicle ECUs according to AUTOSAR methodology, and the toolchain used in the development process, and gives a brief overview about testing AUTOSAR application.

Content

Abstract	1
Content.....	2
List of Figures.....	3
List of Abbreviations	4
1 Introduction	5
2 AUTOSAR Architecture.....	6
2.1 Layers of Basic Software.....	7
2.2 Functional Groups of Basic Software	8
3 Basic Terms	9
3.1 Software Components.....	9
3.2 Ports and Port Interfaces.....	10
3.3 Internal Behavior of Software Component.....	10
4 AUTOSAR Methodology	12
4.1 System Level Design.....	12
4.2 ECU Level Design	13
4.3 Component Level Design	14
5 Development Tools	15
6 Application Test.....	17
7 Conclusion	19
Bibliography.....	20

List of Figures

Figure 1: AUTOSAR Architecture.....6

Figure 2: Layers of BSW7

Figure 3: Functional Groups of BSW8

Figure 4: SWC.....11

Figure 5: VFB12

Figure 6: ECU Level Design13

Figure 7: Workflow of Application Development14

Figure 8: Development Tools15

Figure 9: Toolchain Example.....16

Figure 10: Virtual Validation Platform18

List of Abbreviations

API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software
COTS	Commercial off the shelf
E/E	Electrical/Electronic
ECU	Electrical Control Unit
HiL	Hardware in Loop
I/O	Input/output
ISS	Instruction Set Simulator
MCAL	Microcontroller Abstraction Layer
MiL	Model in Loop
OEM	Original Equipment Manufacturer
OS	Operating System
PC	Personal Computer
PiL	Processor in Loop
PPort	Provide Port
RPort	Require Port
RTE	Runtime Environment
SiL	Software in Loop
SWC	Software Component
VFB	Virtual Functional Bus
XML	Extensible Markup Language

1 Introduction

AUTOSAR stands for **automotive open system architecture**. It is a world development partnership in automotive domain founded in 2003 by leading automotive OEMs and suppliers. The aim of the partnership is to create a reference architecture and development standard for automotive software to master the increasing complexity of software in modern vehicles.

Before AUTOSAR, the development of software for ECUs has been problematic. One of the main concerns is the software dependency on the underlying hardware, since the software communicates directly with the ECU hardware. So, in case of changing the underlying hardware, the software needs to be completely modified. Another drawback is that the software for each ECU used to be developed separately. Although this seems to be the normal case as each ECU runs different executable, but it is not, because there are basic functionalities that are implemented repeatedly for each ECU but they are approximately the same.

AUTOSAR comes to solve these problems. AUTOSAR masters the software complexity and makes the development process more manageable by making the development of the system functionality independent of hardware. AUTOSAR makes it also possible to transfer the application software from one platform to another and avoids redevelopment of similar software components with the help of standardization in software development.

2 AUTOSAR Architecture

AUTOSAR architecture describes how software for ECUs should look like. AUTOSAR is based on the concept of separation between infrastructure and application.

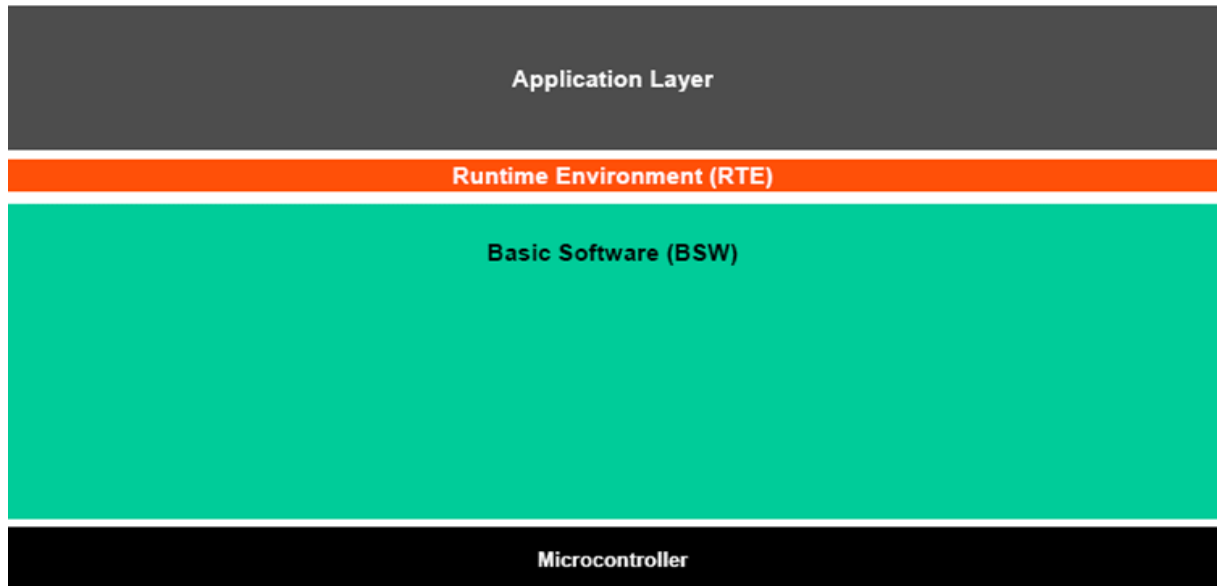


Figure 1: AUTOSAR Architecture

Figure 1 shows how the ECU software looks like according to AUTOSAR. The upper layer is the application layer which implements ECU application functionality e.g. engine control.

The lower layer is the infrastructure or basic software (BSW). The basic software implements basic functionalities or services which are needed by any ECU application software regardless of the specific application functionalities of the individual ECUs e.g. operating system functionalities or bus access.

The application layer can only talk to the basic software through fixed and well defined interfaces. These interfaces are defined by AUTOSAR. The advantage achieved here is that the implementation of BSW may change because BSW is hardware dependable but the services it delivers to the application layer are always the same. So, whenever the hardware changes, only the implementation of BSW changes. This makes the application layer hardware independent. The runtime environment (RTE) realizes the communication between the software components of the application layer and between the software components and BSW. This should be more clear in the next chapter.

2.1 Layers of Basic Software

The basic software itself is subdivided into three layers as shown in Figure 2.

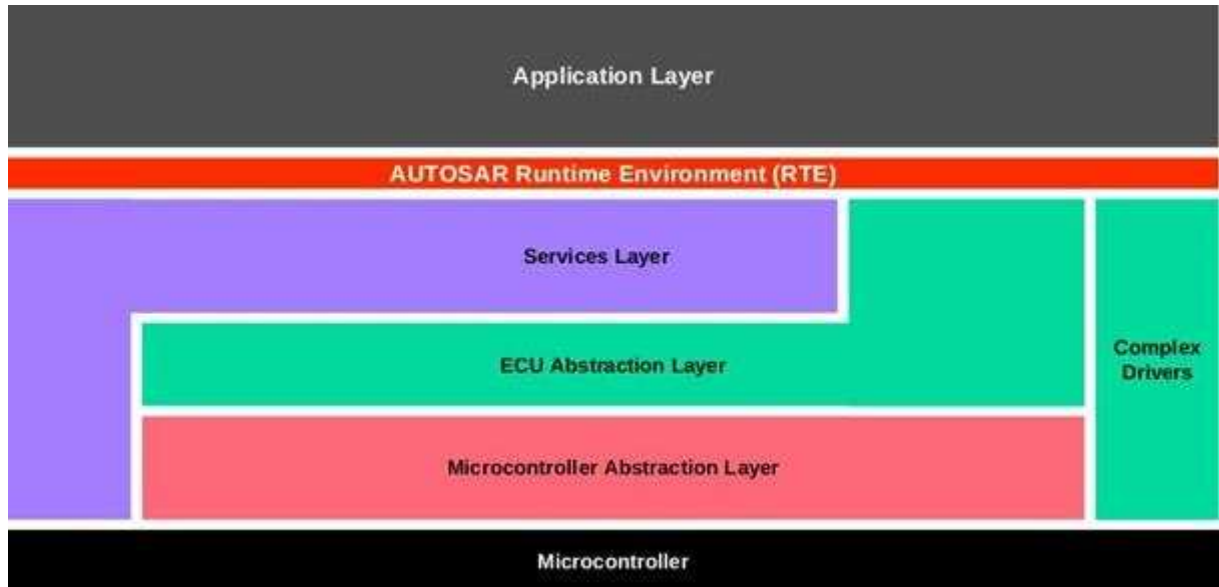


Figure 2: Layers of BSW

The lowest layer is the microcontroller abstraction layer (MCAL). This layer contains drivers which directly access internal peripherals of the microcontroller which makes this layer very hardware dependent. This layer makes the upper layers independent of the microcontroller.

The layer in the middle is ECU abstraction layer. This layer contains also drivers for external peripherals which exist on the ECU but outside the microcontroller. It also interfaces the drivers of the microcontroller abstraction layer. So, this layer offers an API to access all the peripherals which are located inside the microcontroller or outside on the ECU. So, this layer depends on the ECU hardware and makes its upper layer hardware independent.

The most upper layer in basic software is the services layer. This layer is hardware independent. It provides basic services for the application layer and basic software modules.

These layers communicate with each other through interfaces. So, the implementation of one layer may change without influencing on the other layers.

Complex driver area is ECU and microcontroller dependent. It is used to implement special purpose functionality which is not defined by AUTOSAR e.g. device driver, which does not exist in microcontroller or ECU abstraction layers.

2.2 Functional Groups of Basic Software

The layers of the basic software are further divided into functional groups as shown in Figure 3.

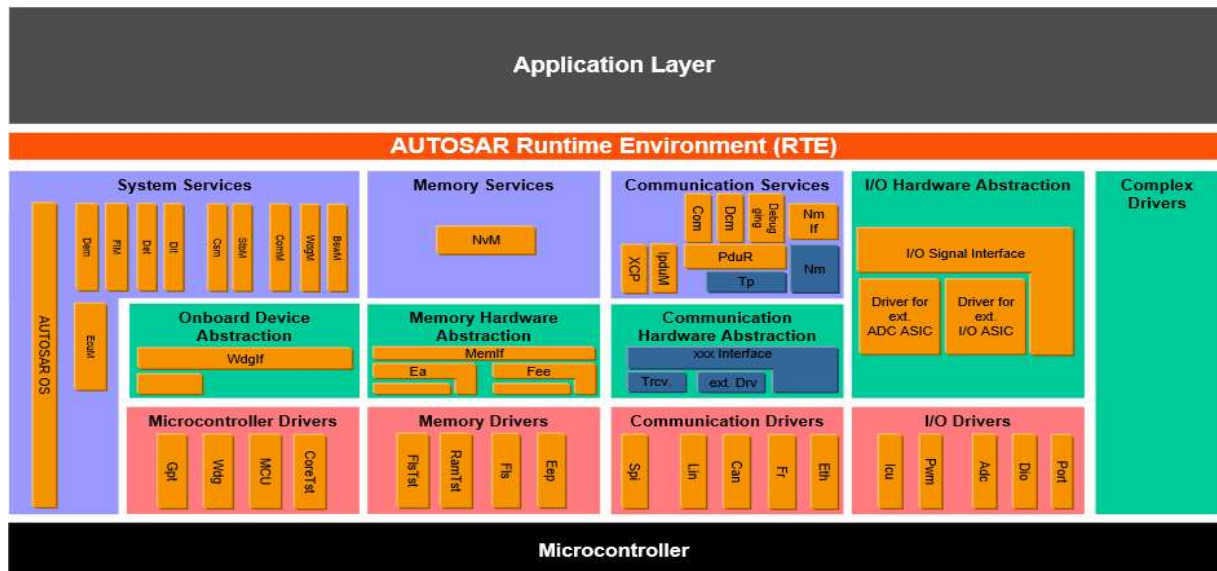


Figure 3: Functional Groups of BSW

These functional groups are:

- I/O functional group is responsible for accessing sensors, actuators, and ECU onboard peripherals.
- Communication functional group is responsible for accessing the network system and buses.
- Memory functional group is responsible for accessing internal and external non-volatile memory.
- System functional group provides services like operating system, timers, and watchdog management.

The idea here is that each functional group has a set of software components of high modularity which means that they are highly decoupled. So, in case that some of these modules are not needed for specific ECU, they can be removed from the basic software of that ECU which makes the size of the final code smaller.

3 Basic Terms

The ECU functionality is implemented in the application layer as a set of software components that communicate with each other in the same ECU and with other software components in the other ECUs. AUTOSAR application development is a model-based design. In this chapter, some basic terms regarding AUTOSAR elements that are used in system design are to be introduced.

3.1 Software Components

In AUTOSAR, there are two main types of software components (SWCs):

- Atomic software component: This type of software components cannot be split into further software components.
- Composite software component: it contains a subset of other software components that could be atomic or composite. It serves as an aggregate which makes it possible to arrange the system design in hierarchy.

The following is the most important software component types that are atomic:

- Application software component type: It implements part of the software application e.g. controlling algorithm. It may read a sensor data or write data to actuator with the help of sensor/actuator software component type.
- Service software component type: It provides services to the application layer through a standardized AUTOSAR interfaces. It communicates directly with basic software.
- Sensor/Actuator software component type: It is used in the application layer to read data from sensor or to write data to actuator. This type of software component is hardware dependent. It is used for specific ECU with specific sensor or actuator. It communicates directly to I/O hardware abstraction layer via ECU abstraction layer software component type.
- ECU abstraction software component type: This type of software components is part of the basic software, specifically from I/O functional group of the ECU abstraction layer. It is connected to sensor/actuator software components and provides access to ECU specific I/Os.
- Complex device driver software component type: It is used to implement functionality that is not AUTOSAR compliant. It can communicate with application components and with modules of basic software.

3.2 Ports and Port Interfaces

Software components communicate with each other through ports. Ports serve as interaction points.

There are two types for ports:

- Provide port (PPort): This type of ports is used by a software component to send data or to request a service.
- Require port (RPort): This type of ports is used by a software component to receive data.

Each port must have one and only one interface. The interface is a contract that declares the data elements or operations that are provided or required by the port. It defines also the communication mechanism the port uses.

The most two important types of port interfaces are:

- Sender-Receiver port interface: With this interface, the communication is asynchronous. There are one sender and possibly many receivers. When the sender sends a data, each receiver decides whether to receive the data or not.
- Client-Server port interface: The server waits for requests from the clients, and whenever it receives a request, it processes it and returns the result to the client.

3.3 Internal Behavior of Software Component

The implementation of the functionality of a software component is described by its internal behavior. The internal behavior of the software component contains one or more runnable entities. The runnable entity represents a C function. Runnable entities should be configured by specifying which data elements of the ports, the runnable entity can see and deal with, and by defining how the runnable entity should be activated. The runnable entity can be activated periodically or event triggered.

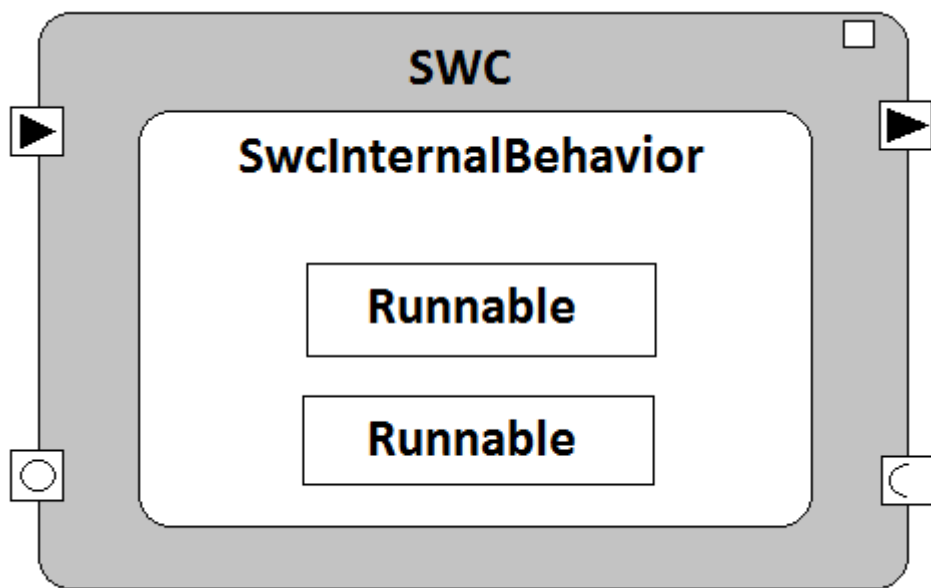


Figure 4: SWC

Figure 4 illustrates the contents of the application software component. The internal behavior contains also one implementation which describes the location of a C file which should be used by the compiler. The C file can be created by a script and then coding details should be manually provided, or it can be generated from a functional block model design by model-based design tools like TargetLink.

4 AUTOSAR Methodology

AUTOSAR application development is a distributed development process. Many players are involved. Each OEM has a main supplier (tier 1) which supplies the OEM with hardware e.g. ECUs and contributes in the software development process, some software companies sell COTS software components. The OEM itself contributes in the development process specially when it wants to preserve its intellectual property. The idea here is that there are different participants with different roles in the development process (different tools are also used). Thus, there is a necessity for software development approach to be followed by everyone with standardized information exchange.

AUTOSAR methodology is a software development approach that defines the workflow of the development process in three different levels (system level, ECU level, and component level) with standardized information exchange through XML files.

4.1 System Level Design

In this level, the software architecture is developed. The entire set of vehicle functions is described as a system of software components. Each software component is described by an XML file called *software component description*. At this level, it is not necessary to have implementation for SWCs. Only description is sufficient.

The entire communication between software components is handled conceptually by virtual functional bus (VFB). As shown in figure 5, VFB describes the logical communication between all SWCs of the system regardless at which ECU, each SWC will be allocated, which means that VFB is independent of the target technology.

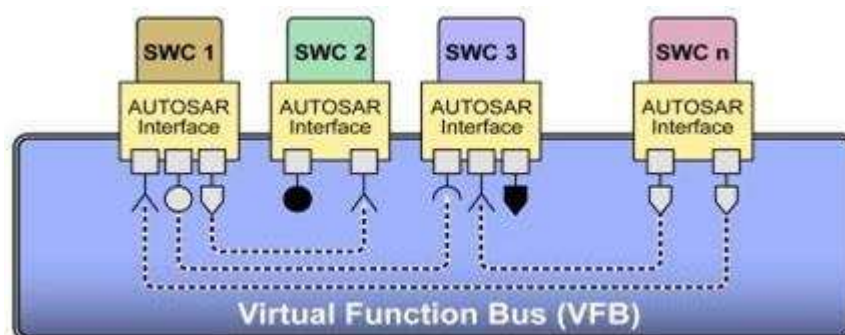


Figure 5: VFB

VFB is important to validate the system design at early stage. The hardware topology and communication network (bus signals and communication metrics e.g. dbc, Fibex, Idf files) are also defined in this level.

After the whole system is described, software components are mapped to ECUs.

The output of the system design level is an XML file called *System Description*.

4.2 ECU Level Design

The workflow in this level is repeated for each ECU. Tier 1 normally does it. From the *System Description* file, relevant information for one ECU is extracted. This extraction is an XML file called *ECU Extract of System Description*. This file contains information about software components mapped to that ECU, hardware resources, and communication information. Based on this file, basic software, and runtime environment (RTE) are configured for that ECU.

The configuration of basic software adapts the basic software for specific ECU needs, like selecting the basic software modules needed and configuring them, the runnable entities of the ECU software components are mapped to tasks for the operating system (OS), scheduling algorithm for tasks and their priorities are defined, data elements of the ports that are connected to ports of software components located in other ECUs, are mapped to bus signals.

The output from the configuration is XML file called *ECU Configuration Description*. Code generators for BSW and RTE use this file to generate implementation files for both BSW and RTE for that specific ECU.

RTE is implementation of the VFB in one ECU. So, all RTE layers in all ECUs are implementation for the conceptual VFB. RTE differs from one ECU to another.

To generate the executable file for the ECU, the implementation files for RTE and BSW with the implementation files for the SWCs of that ECU are together compiled by the target compiler. Figure 6 illustrates the work done in the ECU level.

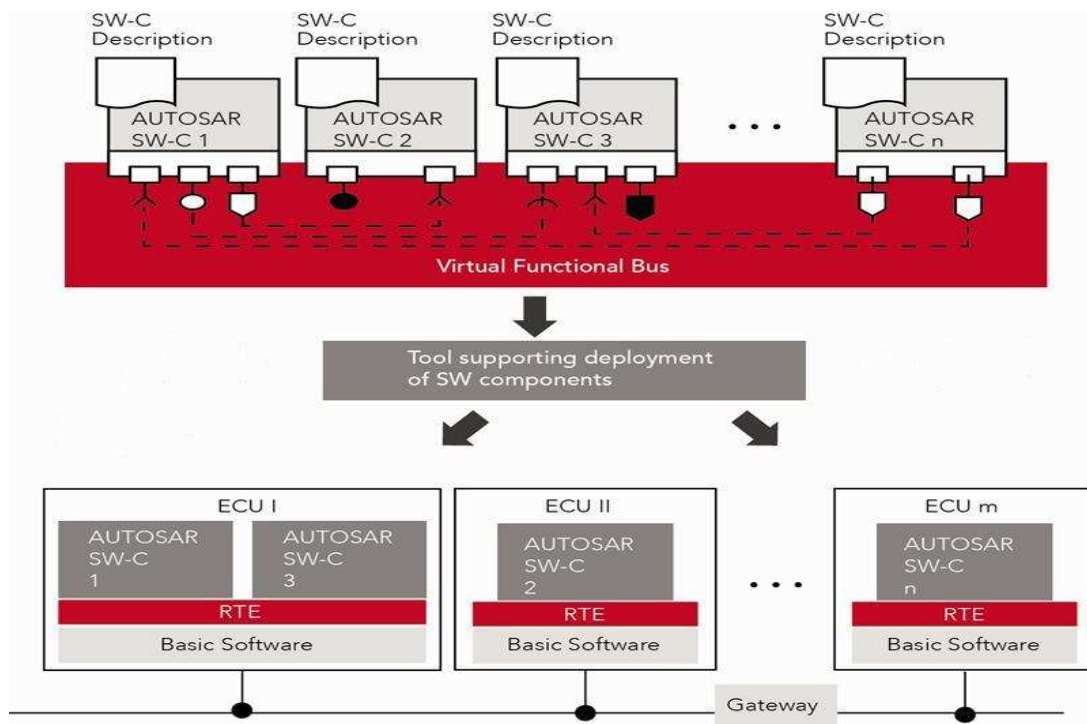


Figure 6: ECU Level Design

4.3 Component Level Design

In this level, functionality of software component is implemented. The implementation of the software component can be hand-coded or generated from a model design by modelling tools. Using such a tool, the functionality of the software component is described by diagram of defined functional blocks, then the code is generated. Simulink TargetLink is a good example, it is an AUTOSAR compliant tool which takes a software component description as an input, then the tool builds a work frame from the description that can be then further completed to completely describe the functionality of the software component from which the code is generated. With this tool, it is also possible to develop a software component from the scratch with its implementation, and then export the description and implementation files so that system design tools can import them.

Figure 7 illustrates the workflow of the development process for AUTOSAR application according to AUTOSAR methodology in the three levels combined.

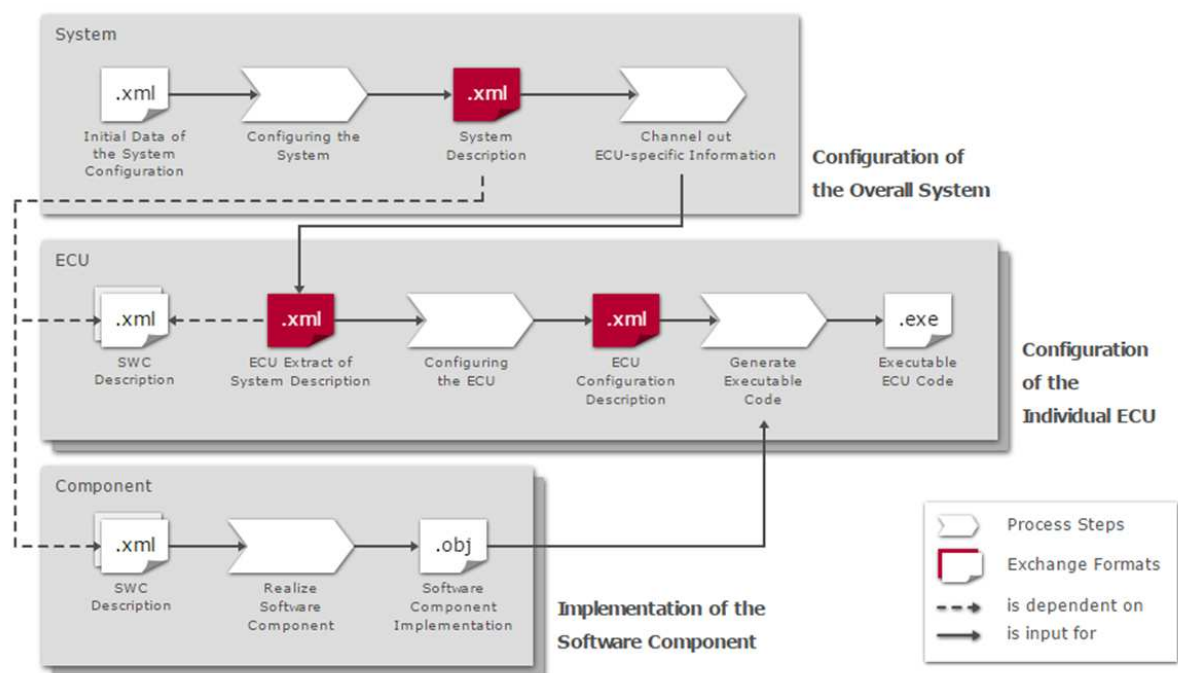


Figure 7: Workflow of Application Development

5 Development Tools

There are different tools used to create AUTOSAR ECU software. these tools can be in general classified as Figure 8 illustrates as follows:

- Tools for system design: They are used to do the work done in system level design which is described in the previous chapter. These tools can export the XML files *System Description* and *ECU Extract of System Description*.
- Tools for configuring BSW and RTE: They are used to do the work done in ECU level design as described in the previous chapter. These tools can export the XML file *ECU Configuration Description*.
- Code generators for BSW and RTE: Based on *ECU Configuration Description* file, BSW and RTE implementation files are generated.

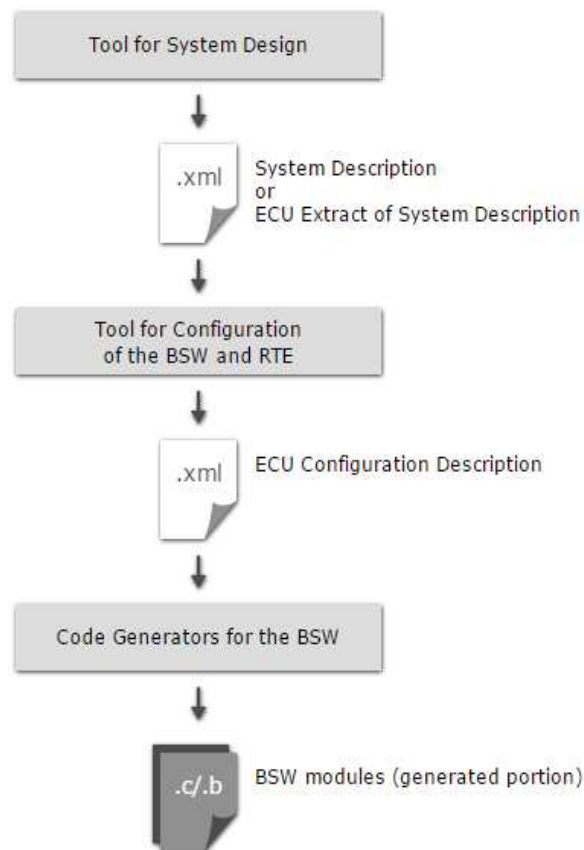


Figure 8: Development Tools

Model-based design tools may also be used to generate implementation files for SWCs. Target compilers are of course necessary to generate the executable files for the ECUs.

These tools exchange the design and configuration data with the help of XML files. These files are standardized and well defined by AUTOSAR. The standardization is very essential, it is like everybody speaks the same language. Tools may come from different manufacturers and used in the same development project. Figure 9 shows a famous toolchain used for AUTOSAR development.

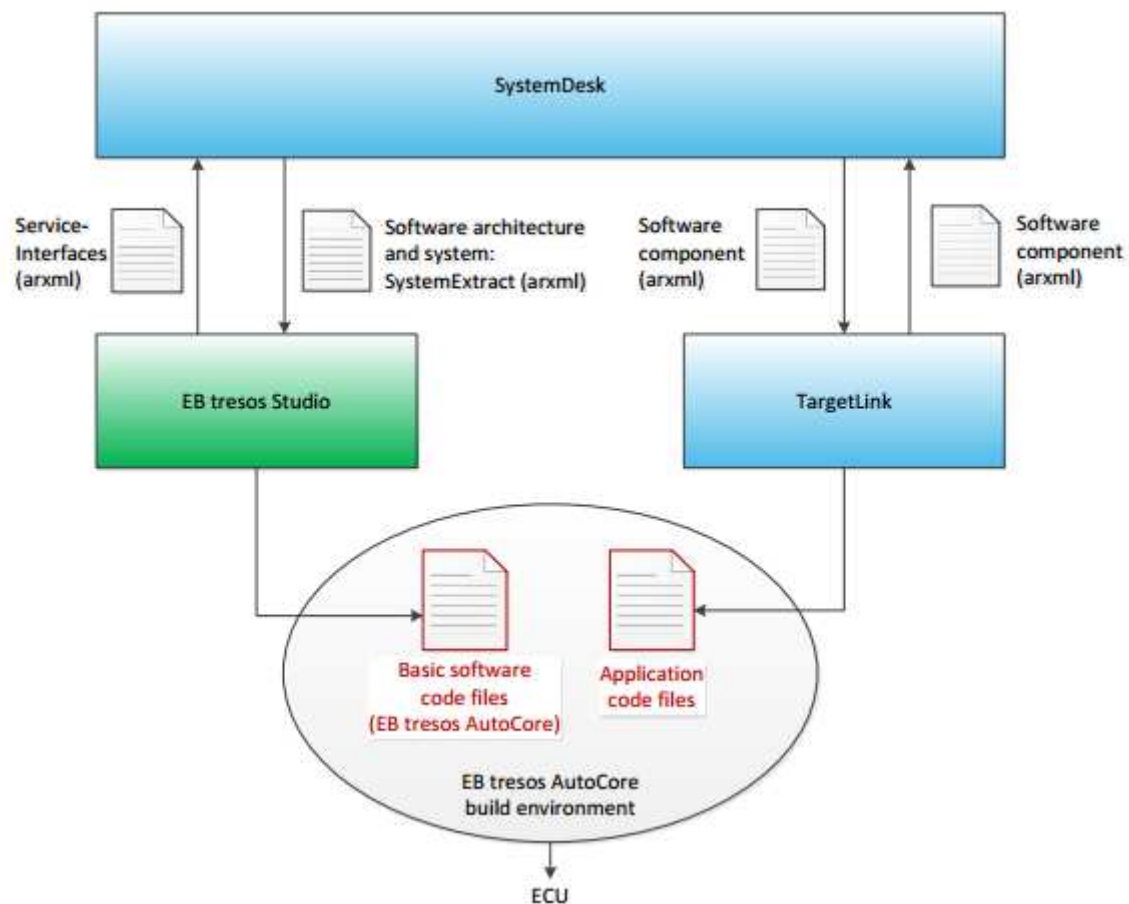


Figure 9: Toolchain Example

SystemDesk from dSpace company used for system design, EB tresos Studio from electrobit company used to configure BSW and RTE and to generate their code files, and TargetLink from dSpace company for modelling software components and generating their codes. Normally Windriver compiler is used for code compilation.

6 Application Test

Electronic systems in automotive domain is safety critical systems. Software plays crucial role in system reliability. Because of that, test of software is very important. The main idea is to validate the software design as early as possible to reduce development time. Late discovery of software faults may lead to a lot of modifications to the software design which costs a lot of time and money.

Testing of embedded systems can be classified into four types depending on the availability of the target platform as follows:

- Model in loop (MiL): With this type, only a functional model is available for system software. the model can be simulated for functional correctness. TargetLink supports this kind of testing.
- Software in loop (SiL): With this type, the software implementation is available. The software is adapted and compiled to run in PC rather than the target platform.
- Processor in loop (PiL): With this type, the target platform is simulated. Thus, the target compiler is used to compile the software.
- Hardware in loop (HiL): With this type, the software runs in the target platform and the environment is simulated e.g. the software runs in ECU and the environment is simulated by writing and reading signals from the connected bus.

A very important testing method for AUTOSAR application is virtual validation platform. In virtual validation platform, the software is tested in PC rather than real ECU. Most of AUTOSAR system tools support this method e.g. SystemDesk enables developers to generate virtual ECUs. The BSW and RTE code is generated for these virtual ECUs in two variants as shown in Figure 10.

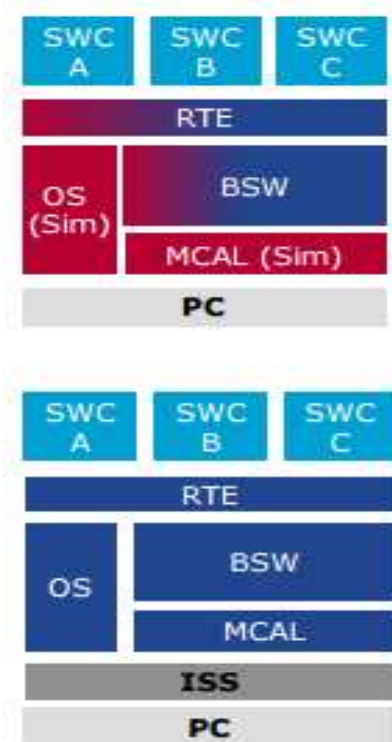


Figure 10: Virtual Validation Platform

The first variant is software in loop testing. The code for BSW is adapted to run in PC, because microcontroller abstraction layer and OS depend on the target platform. Thus, the same code used for real ECU cannot be used to run in PC. It is important here to know that the aim here is to test the application layer not the basic software.

The second variant is processor in the loop testing. In this variant, the code for BSW and RTE is not changed. It is the same used for real ECU because the target platform is simulated i.e. instruction set simulator (ISS) to simulate the functionality of the microprocessor in the real ECU. So, here the target compiler is used.

When the virtual ECU is ready, it can be executed and different tools can be used for configuration purposes by accessing internal variables of the virtual ECU and for visualizing the simulation results. Examples for such tools are dSpace ControlDesk or Vector CANoe.

7 Conclusion

AUTOSAR masters the growing complexity of software in the automotive domain, and enables reusability of similar software components with the help of standardization in the AUTOSAR software development process.

The layered AUTOSAR architecture makes the development of software application independent of underlying hardware by the separation between the application and the basic software.

AUTOSAR methodology is a software development approach that defines the workflow of AUTOSAR application development process. AUTOSAR methodology defines no roles, it just describes the main steps in the development process and makes it more manageable.

AUTOSAR application development is a distributed development process i.e. many participants and different development tools. Thus. Standardization is very important. AUTOSAR defines a set of standardized XML files used for information exchange.

AUTOSAR still cannot automate code generation for software components. It should be hand coded or generated with the help of model-based design tools like TargetLink.

Bibliography

- [1] Matthias Wernicke: AUTOSAR on its Way to Production,
https://vector.com/portal/medien/cmc/press/Vector/AUTOSAR_Elektronikpraxis_200802_PressArticle_EN.pdf
- [2] AUTOSAR 4.2 Specification: Layered AUTOSAR Architecture,
https://www.autosar.org/fileadmin/files/releases/4-2/software-architecture/general/auxiliary/AUTOSAR_EXP_LayeredSoftwareArchitecture.pdf
- [3] Vector: Introduction to AUTOSAR,
https://elearning.vector.com/vl_autosar_introduction_en.html
- [4] dSpace: SystemDesk - EB tresos Studio - TargetLink Workflow Descriptions,
https://d23rjziej2pu9i.cloudfront.net/wp-content/uploads/2014/11/28025450/SystemDesk_EB_tresos_Studio_TargetLink_Application_Note_EN.pdf
- [5] Vector: Integration und Test virtueller AUTOSAR Steuergeräte,
https://vector.com/portal/medien/cmc/events/commercial_events/VectorCongress_2012/VeCo12_7_ECUtesting_3_Schwager_Lecture.pdf